Architecture and GitHub Repository Guide: PIDray Journal Baseline and Occlusion-Aware Bilayer Modeling

1. How to Use This Guide

This guide answers four implementation questions: *Which exact model is the baseline? Which upstream projects did it start from? Where is each architectural change implemented? In what order should the repository be read and executed?* It separates paper descriptions from released-code behavior. File names and settings refer to the public default branches reviewed for this study; record a commit hash when beginning the actual reproduction.

2. PIDray Journal Baseline: Model Identity and Lineage

Official source: `github.com/lutao2021/PIDray`. The full journal baseline is not a Python class named "PIDrayNet." It is assembled by `cascade_mask_rcnn_r101_with_R0R1.py` using a modified, vendored MMDetection 2.25 codebase. The PIDray-specific additions are AttFPN and an R_0 confidence branch inserted inside the shared two-stage detector. SDANet is an earlier comparison method, not the name of this configuration.

Origin	Project / version	What PIDray uses or changes
Detection framework	`MMDetection 2.25.0`	Cascade R-CNN, RPN, RoI Align/heads, mask head, datasets, training runner, COCO evaluator; repository vendors and modifies it.
Vision utilities	`MMCV 1.5.2`	Convolutions, NMS, runner utilities, and the `ContextBlock` used in AttFPN.
Backbone weights	`torchvision 0.11`	ImageNet-pretrained ResNet-101 initialization.
Confidence decoder	`Query2Label` / DETR style	One learned query cross-attends to a pyramid feature; the local source header records Q2L lineage.
Evaluation	`COCO API`	Box and segmentation AP/AR over COCO IoU thresholds.

3. PIDray Architecture: Step-by-Step Tensor Path

Step	Component	Released setting	Output / purpose
1	Preprocess	Fit within 500×500, keep ratio; ImageNet normalize; pad 32	Variable spatial size divisible by 32; flip 0.5 during training.
2	ResNet-101	Bottleneck stages $[3, 4, 23, 3]$; outputs 256/512/1024/2048 channels	C_2–C_5 at strides 4/8/16/32. Stem and first residual stage frozen; BatchNorm in eval mode.
3	FPN base	Lateral 1×1, top-down addition, output 3×3, max-pooled extra level	Five 256-channel maps P_2–P_6 at strides 4/8/16/32/64.
4	AttFPN	Each target level gathers all five resized levels	Channel-, spatial-, and context-refined five-level pyramid.
5	R_0 branch	P_6 + sine position; one 256-D query; 2 decoder layers, 4 heads, FFN 1024	One prohibited-present logit; BCE-with-logits weight 0.1; route at detached probability > 0.5.
6	RPN	Anchor scale 8, ratios 0.5/1/2 on five strides	Class-agnostic proposals; ratio-1 nominal sizes 32/64/128/256/512 px.
7	Cascade boxes	Three Shared-2FC heads, width 1024; IoU 0.5/0.6/0.7; weights 1/0.5/0.25	Twelve-class scores and progressively refined, class-agnostic boxes.
8	Masks	RoI Align 14×14; four 256-channel convs; class-specific output	28×28 mask target for each positive RoI.

AttFPN in detail. For target level ℓ, finer levels are adaptively max-pooled and coarser levels are nearest-neighbor interpolated. Channel attention uses global average pooling, a $256 \to 128$ bottleneck and five $128 \to 256$ projections; spatial attention uses channel mean/max followed by five 7×7 convolutions. Both normalize with a softmax across source levels. Their fused maps pass through `ContextBlock(256, 1/16)` and are added residually to P_ℓ.

Routing nuance. Training sends only predicted-positive images to RPN/RoI, while R_0 learns on all images. The printed detector loss is preserved, but its gradient is scaled by the predicted selected fraction. At public-code inference, RPN/RoI run first and low-R_0 scores are then zeroed for the first batch element; this is output suppression, not computational early exit.

4. PIDray Repository: Read, Run, and Verify

4.1 Source map in the recommended reading order

Order	Official file / link	What to learn or verify
1	`README / repository root`	Dataset and checkpoint links; train/test commands; note that the README command and paper hardware count are not identical.
2	`configs/...R0R1.py`	Backbone, AttFPN, RPN, cascade and mask settings; paths; transforms; optimizer; 12 epochs; box/mask evaluation.
3	`mmdet/models/necks/attfpn.py`	Five-source resizing, channel/spatial level softmax, context block, residual fusion.
4	`mmdet/models/detectors/two_stage.py`	R_0 construction, binary target, loss, hard routing, gradient scaling, and test-time score suppression.
5	`confidence_branch/transformer.py`	Two-layer decoder and removed query self-attention; Q2L/DETR-style implementation.
6	`utils/positional_encoding.py`	Normalized 2-D sine/cosine encoding supplied to the confidence branch.
7	`mmdet/datasets/custom.py`	Empty-image filtering block is disabled, retaining normal baggage images.
8	`requirements.txt` and `tools/`	Legacy package evidence and entry scripts. The listed ROCm torchaudio is inconsistent with CUDA wheels and is unnecessary here.

4.2 Minimal exact-reproduction sequence

1. **Pin, do not modernize.** Record the official commit. Build an isolated legacy environment around PyTorch 1.10 + CUDA 11.1, torchvision 0.11, `mmcv-full` 1.5.2, and the vendored MMDetection. Confirm `mmdet.__file__` points inside the clone.
2. **Manifest PIDray.** Record annotation hashes, 12-class ID/name order, positive/normal counts, train and Easy, Hard, and Hidden membership, invalid polygons, and representative rendered samples.
3. **Change paths only.** Copy the official config and edit dataset/checkpoint/work-directory paths. Do not alter model or optimizer before the released checkpoint test.
4. **Evaluate official weights.** Run `bbox` and `segm`; save predictions and subset metrics. A large mismatch means the data/evaluator integration is not ready for training.
5. **Smoke train.** Use a train-derived subset containing positive and normal images. Log all losses, R_0 logits, selected fraction, RPN proposal recall, and boxes/masks.
6. **Full train and freeze.** Preserve effective batch 16 where possible; record any learning-rate/batch change. Select on a train-derived validation split, freeze settings, then test Easy/Hard/Hidden.

Official command forms are `sh tools/dist_test.sh CONFIG CHECKPOINT N -out RESULT -eval bbox segm` and `sh tools/dist_train.sh CONFIG N`. Treat them as entry patterns, not a complete reproducibility record.

4.3 High-risk details to test explicitly

Risk	Required verification
Normal images silently removed	Count empty-target samples observed by the loader; test the local modified `custom.py`.
Test data used for development	The supplied val/test path is shared; create validation only from training and never tune on Easy/Hard/Hidden.
Gate hides missed positives	Report positive-image R_0 false-negative rate and results before/after score suppression.
Claimed speedup unsupported	Benchmark released code as written; call a true early exit a new variant.
Batch gate bug/assumption	Use batch-one exact inference or patch all entries and label the patch.
Paper/README/config differ	Record actual GPUs, per-GPU and global batch, LR, checkpoint and commit rather than copying prose.

4.4 Best starting modifications after reproduction

For the proposed study, instrument before modifying: (i) R_0 errors on positive/normal images; (ii) RPN recall by size and difficulty; (iii) unmatched high-confidence proposals on normal images; and (iv) mask leakage at overlaps. If background proposals dominate, add a feature/RPN or RoI hard-negative component while keeping R_0 unchanged for the primary ablation. This creates a mechanism distinct from the existing image-level screen.

5. Bilayer + SAM: Model Identity, Lineage, and Architecture

Official source: `github.com/Ryh1218/Occ`. The repository is a modified framework rather than a small plug-in. Its README explicitly credits `RSPrompter` and MMDetection. RSPrompter supplies the Mask R-CNN/SAM learned-prompt starting point; Occ adds the occlusion dataset handling, the second decoder, its loss, and residual guidance.

Layer of lineage	Source	Role in Occ
Application base	`RSPrompter release`	Automatic SAM prompts from RoI features; `RSPrompterAnchor` Mask R-CNN structure.
Detection framework	`MMDetection 3.2.0`	Vendored RPN, RoI heads, datasets, transforms, runner and evaluator.
Backbone wrapper	`MMPretrain 1.1.1`	ViT-SAM encoder implementation and registry integration.
Foundation model	`Meta SAM` / `facebook/sam-vit-large`	ViT-L image encoder, prompt/mask decoder weights; Hugging Face binary is converted locally.
Adaptation	`PEFT LoRA`	Trainable low-rank adapters on encoder `qkv` projections while base weights stay frozen.

6. Exact Forward Path in the PIDray Configuration

1. **Image embedding.** A 512×512 X-ray image enters SAM ViT-L: patch 16, width 1024, depth 24, 16 heads, window 14, global-attention blocks 5/11/17/23. The neck outputs one $256 \times 32 \times 32$ feature.
2. **Parameter-efficient adaptation.** PEFT injects rank-16 LoRA into `qkv` (alpha 32, dropout 0.05, no bias). Base encoder weights are frozen; adapters train. The released `decoder_freeze=False` leaves both mask decoders trainable.
3. **Proposal pyramid.** `OcFeatureAggregator` is an identity for the single feature. `RSSimpleFPN` uses transposed convolution/identity/pooling paths to generate five 256-channel levels with effective strides 4/8/16/32/64.
4. **RPN and boxes.** Each level uses anchor scales 4 and 8 with ratios 0.5/1/2 (six anchors per location). RoI Align 7×7 and a Shared-2FC head (width 1024) predict 12 classes and class-specific box regression.
5. **Learned prompts.** Positive RoI features pass through an MLP and sine/cosine transformation to produce five sparse prompt embeddings compatible with SAM.
6. **Occluder decoder.** `mask_decoder_bo` predicts the overlapping occluding-object region and returns refined image and point embeddings. The custom target is carried as `bg_object_segmentation` in JSON and packed as `bo_masks`.
7. **Target decoder.** Refined embeddings are added residually to the original image/prompt embeddings before the second `mask_decoder` predicts the target item mask. Training adds $0.25L_{\text{occluder}}$ to the ordinary target-mask loss.

Decoder initialization. `convert_model_weights.py` deep-copies every original `mask_decoder.*` tensor to a parallel `mask_decoder_bo.*` key. Both branches therefore begin from the same SAM decoder weights; skipping conversion leaves the new branch uninitialized or incompatible with the expected checkpoint.

Data construction. The paper says PIDray-A/PIXray-A occlusion areas were manually annotated. The optional `convert_occlusion_annotation.py` is an automatic custom-data approximation: it examines object pairs with bounding-box IoU above 0.05, clips other objects' true masks inside the target box, and keeps polygons above a pixel-area threshold. Visually audit these outputs and never label them as the manual release without verification.

6.1 Released PIDray training protocol

Item	Released setting
Environment	Python 3.10; PyTorch 2.1.2 + CUDA 11.8; torchvision 0.16.2; MMCV 2.1.0; Transformers 4.38.1; PEFT 0.8.2; NumPy 1.24
Transforms	Flip 0.5; random resize around 512 with ratio range 0.1–2.0; random 512×512 crop; keep negative crops; transform target and occluder masks together
Optimization	AdamW 10^{-4}, weight decay 0.05, AMP; 50-iteration warm-up from factor 0.001; cosine minimum 10^{-7}; 50 epochs
Batch	Config: 2 images/GPU, 8 workers, no declared accumulation. Paper: batch 64 on one A800 80 GB. Preserve this discrepancy in the experiment record.

7. Bilayer Repository: Read, Run, and Verify

7.1 Source map in the recommended reading order

Order	Official file / link	What to learn or verify
1	`readme.md / repository root`	Exact environment, SAM download/conversion, PIDray-A/PIXray-A links, train/test commands, checkpoints, RSPrompter lineage.
2	`configs/occlusion/pid.py`	ViT-L/LoRA, FPN, RPN/RoI heads, classes, transforms, optimizer, schedule, and data paths.
3	`configs/occlusion/base.py`	Base dataset/evaluator and runtime inheritance used by the shorter PIDray configuration.
4	`mmdet/occlu/models.py`	RSPrompter/SAM wrappers, image encoder, identity aggregator, simple FPN, prompt machinery.
5	`mmdet/occlu/occlu.py`	Custom dataset/transforms, `bo_masks`, bilayer RoI/mask head, two decoder calls, residual guidance and losses.
6	`convert_model_weights.py`	Duplicates SAM decoder tensors for the occluder branch.
7	`convert_occlusion_annotation.py`	Optional heuristic custom-data conversion; not proof of equivalence to manual PIDray-A.
8	`tools/` and `RSPrompter release`	Execution scripts and the architectural starting repository that Occ extends.

7.2 Minimal exact-reproduction sequence

1. Pin the Occ commit and use its vendored MMDetection/MMPretrain rather than separately installing a conflicting modern version. Build the documented CUDA 11.8 environment and verify package import paths.
2. Obtain official PIDray-A and record its annotation hash/counts. If using the converter instead, assign a new dataset name and audit image, instance, and occluder-mask counts plus overlays.
3. Download `facebook/sam-vit-large` with a PyTorch binary, update paths in the converter, create the duplicated-decoder checkpoint, and inspect missing/unexpected keys during load.
4. Change only dataset, checkpoint and output paths in `pid.py`. Evaluate the released checkpoint before training.
5. Run one forward/backward batch. Verify trainable LoRA parameters; trainable target and occluder decoders; nonzero RPN/box/target-mask/occluder-mask losses; and synchronized geometric transforms for both mask types.
6. Run a small subset, then full training. Record actual per-GPU/global/effective batch and do not claim paper-exact optimization if it differs from batch 64.

7.3 High-risk checks and fair use

Risk	Required verification
"Frozen SAM" is oversimplified	List trainable parameters: base encoder frozen, LoRA qkv adapters trainable, both decoders trainable in released config.
Wrong checkpoint structure	Confirm both `mask_decoder` and `mask_decoder_bo` keys load from converted weights.
Occluder masks misaligned	Render before/after flip, resize and crop; verify `bo_masks` correspond to each target instance.
Manual/generated data conflated	Report dataset source and converter rules; do not compare as the same annotation release.
Hidden performance overclaimed	Report Easy/Hard/Hidden: published Bilayer improves Easy/Hard but remains below PIDrayNet on Hidden.
Resource mismatch hidden	Record batch, accumulation, AMP, memory, schedule and hardware; label deviations.

8. Which Repository Should Be the Starting Point?

Goal	Start here	Reason
Primary thesis baseline	`PIDray official`	Direct match to original PIDray, normal images, boxes/masks, and Easy/Hard/Hidden. Reproduce first.
Bilayer exact reproduction	`Occ official`	Contains the actual dual decoder, occlusion data path, weight conversion and released configs.
Understand Bilayer inheritance	`RSPrompter release`	Explains the learned-prompt Mask R-CNN + SAM structure inherited by Occ; do not substitute its config for Occ.
Modern implementation	New declared branch after exact runs	Porting PIDray to MMDetection 3.x or changing SAM/backbone is a new implementation, useful only after reference metrics are secured.

Recommended order for this project: PIDray released checkpoint → Cascade Mask R-CNN anchor → full PIDray reproduction → error taxonomy → one background-confusion component → PIDray final test → Occ/PIDray-A feasibility and reproduction. This preserves a clear research contribution and prevents the heavier SAM track from becoming an unstructured replacement for the primary baseline.

Primary Papers

L. Zhang et al., *PIDray: A Large-Scale X-ray Benchmark for Real-World Prohibited Item Detection*, IJCV, 2023, `https://arxiv.org/abs/2211.10763`.

Y. Ren et al., *Prohibited Items Segmentation via Occlusion-Aware Bilayer Modeling*, ICME, 2025, `https://arxiv.org/abs/2506.11661`.